

Doc 7.4 — Computed Properties & Advanced Object Patterns

এই Doc-এ কী শিখব?

এই document-এ আমরা JavaScript Object-এর **dynamic** এবং **advanced patterns** শিখব।

Topics

1. Computed Property Names
 2. Dynamic Property Names
 3. Bracket Notation + Variables
 4. Computed Properties in Object Literals
 5. Dynamic Object Creation
 6. Dynamic Keys from Form Data
 7. Dynamic Keys from API Data
 8. Computed Properties + Shorthand
 9. Computed Properties + Spread
 10. Computed Properties + Destructuring
 11. Immutable Object Update
 12. Dynamic Object Update
 13. Nested Dynamic Object Update
 14. React State-এর Dynamic Key Update
 15. Common Mistakes
 16. Practical Patterns
-

1. Computed Property Names

JavaScript object-এর property name সাধারণত আমরা সরাসরি লিখি:

```
const user = {  
  name: "Shariar",  
  age: 31  
};
```

এখানে:

```
name
age
```

দুটোই static property name।

কিন্তু অনেক সময় property name আগে থেকে জানা থাকে না।

সেক্ষেত্রে আমরা variable-এর value ব্যবহার করে property name তৈরি করতে পারি।

এটাকেই বলে:

Computed Property Name

Syntax:

```
const key = "name";

const user = {
  [key]: "Shariar"
};
```

Result:

```
{
  name: "Shariar"
}
```

এখানে:

```
[key]
```

মানে:

`key` variable-এর value বের করে property name হিসেবে ব্যবহার করো।

2. Normal Property vs Computed Property

Normal property

```
const user = {  
  name: "Shariar"  
};
```

এখানে `name` সরাসরি property name।

Computed property

```
const key = "name";  
  
const user = {  
  [key]: "Shariar"  
};
```

এখানে `key` evaluate হবে।

```
key  
  ↓  
"name"  
  ↓  
property name
```

তাই final object:

```
{  
  name: "Shariar"  
}
```

3. Dynamic Property Names

Dynamic মানে:

আগে থেকে *fixed* না।

ধরো:

```
const key = "email";

const value = "shariar@example.com";

const user = {
  [key]: value
};

console.log(user);
```

Output:

```
{
  email: "shariar@example.com"
}
```

এখন key পরিবর্তন করলে object-ও পরিবর্তন হবে।

```
const key = "phone";

const user = {
  [key]: "01700000000"
};
```

Result:

```
{
  phone: "01700000000"
}
```

4. Bracket Notation + Variable

Dynamic property access-এর জন্য bracket notation অত্যন্ত গুরুত্বপূর্ণ।

```
const user = {  
  name: "Shariar",  
  age: 31  
};  
  
const key = "name";  
  
console.log(user[key]);
```

Output:

```
Shariar
```

থেয়াল করো:

```
user[key]
```

এটা correct।

কিন্তু:

```
user.key
```

এটা `key` নামের property খুঁজবে।

```
console.log(user.key);
```

Output:

```
undefined
```

কারণ object-এ `key` নামে কোনো property নেই।

5. Dot Notation vs Bracket Notation

Syntax	Meaning
<code>user.name</code>	<code>"name"</code> property
<code>user["name"]</code>	<code>"name"</code> property
<code>user[key]</code>	<code>key</code> variable-এর value অনুযায়ী property
<code>user.key</code>	<code>"key"</code> property

সবচেয়ে গুরুত্বপূর্ণ:

```
user[key]
```

এখানে variable evaluate হয়।

কিন্তু:

```
user.key
```

এখানে literal `"key"` property খোঁজা হয়।

6. Computed Properties in Object Literals

Object তৈরি করার সময়ও computed property ব্যবহার করা যায়।

```
const firstKey = "name";
const secondKey = "age";

const user = {
  [firstKey]: "Shariar",
  [secondKey]: 31
};

console.log(user);
```

Result:

```
{
  name: "Shariar",
  age: 31
}
```

এখানে:

```
[firstKey]
```

হয়ে গেছে:

```
name
```

আর:

```
[secondKey]
```

হয়ে গেছে:

```
age
```

7. Expression ব্যবহার করা যায়

Computed property-এর ভিতরে শুধু variable নয়, expression-ও ব্যবহার করা যায়।

```
const prefix = "user";

const user = {
  [`${prefix}Name`]: "Shariar"
};
```

Result:

```
{
  userName: "Shariar"
}
```

আরেকটি example:

```
const index = 1;

const data = {
  [`item${index}`]: "Laptop"
};
```

Result:

```
{
  item1: "Laptop"
}
```

8. Function Call ব্যবহার করা যায়

Computed property-এর ভিতরে expression থাকলে function call-ও করা যায়।

```
function getKey() {
  return "username";
}

const user = {
  [getKey()]: "Shariar"
};

console.log(user);
```

Result:

```
{
  username: "Shariar"
}
```

অর্থাৎ computed property:

```
[key]
```

এর ভিতরের expression আগে evaluate হয়।

9. Dynamic Object Creation

ধরো আমরা এমন একটা function বানাতে চাই যেটা dynamic key দিয়ে object তৈরি করবে।

```
function createObject(key, value) {  
  return {  
    [key]: value  
  };  
}  
  
const result = createObject("name", "Shariar");  
  
console.log(result);
```

Output:

```
{  
  name: "Shariar"  
}
```

আর:

```
createObject("age", 31);
```

Result:

```
{  
  age: 31  
}
```

10. Dynamic Key + Dynamic Value

দুটোই dynamic হতে পারে।

```
function createProperty(key, value) {  
  return {  
    [key]: value  
  };  
}  
  
const key = "role";  
const value = "developer";  
  
const user = createProperty(key, value);
```

Result:

```
{  
  role: "developer"  
}
```

11. Form Data-এর ক্ষেত্রে Computed Properties

এটা real-world development-এ অনেক গুরুত্বপূর্ণ।

ধরো একটি form:

```
name  
email  
password
```

User যে field change করবে, আমরা সেই field-এর name অনুযায়ী state update করতে চাই।

ধরো:

```
const fieldName = "email";  
const fieldValue = "shariar@example.com";
```

আমরা করতে পারি:

```
const updatedData = {  
  [fieldName]: fieldValue  
};  
  
console.log(updatedData);
```

Output:

```
{  
  email: "shariar@example.com"  
}
```

12. Dynamic Form Object

ধরো:

```
const formData = {};  
  
const fieldName = "name";  
const fieldValue = "Shariar";  
  
formData[fieldName] = fieldValue;  
  
console.log(formData);
```

Result:

```
{  
  name: "Shariar"  
}
```

এরপর:

```
formData["email"] = "shariar@example.com";
```

Result:

```
{
  name: "Shariar",
  email: "shariar@example.com"
}
```

13. React-এ Dynamic Form State

React-এর controlled form-এর ক্ষেত্রে এই pattern অনেক বেশি ব্যবহার হয়।

```
const [form, setForm] = useState({
  name: "",
  email: "",
  password: ""
});
```

ধরো:

```
<input
  name="email"
  value={form.email}
  onChange={handleChange}
/>
```

Handler:

```
const handleChange = (event) => {
  const { name, value } = event.target;

  setForm((prev) => ({
    ...prev,
    [name]: value
  }));
};
```

এখানে সবচেয়ে important অংশ:

যদি:

```
name === "email"
```

তাহলে:

```
{
  ...prev,
  email: value
}
```

যদি:

```
name === "password"
```

তাহলে:

```
{
  ...prev,
  password: value
}
```

একই handler দিয়ে multiple input handle করা যায়।

14. Computed Property + Spread

Computed property এবং object spread একসাথে ব্যবহার করা যায়।

```
const user = {
  name: "Shariar",
  age: 31
};

const key = "role";

const updatedUser = {
  ...user,
  [key]: "developer"
};
```

Result:

```
{
  name: "Shariar",
  age: 31,
  role: "developer"
}
```

এখানে:

```
...user
```

পুরোনো properties copy করছে।

আর:

নতুন dynamic property তৈরি করছে।

15. Existing Property Dynamicভাবে Update

```
const user = {
  name: "Shariar",
  age: 31
};

const key = "age";

const updatedUser = {
  ...user,
  [key]: 32
};
```

Result:

```
{
  name: "Shariar",
  age: 32
}
```

কারণ একই property পরে আসায় নতুন value পুরোনোটাকে overwrite করেছে।

16. Dynamic Object Update Pattern

এই pattern খুব important:

```
const updatedObject = {  
  ...oldObject,  
  [key]: newValue  
};
```

এটার mental model:

```
old object  
    ↓  
spread/copy  
    ↓  
dynamic property  
    ↓  
new object
```

Example:

```
const product = {  
  name: "Laptop",  
  price: 80000  
};  
  
const key = "price";  
const newValue = 75000;  
  
const updatedProduct = {  
  ...product,  
  [key]: newValue  
};
```

Result:

```
{
  name: "Laptop",
  price: 75000
}
```

17. Why This Pattern Is Important in React

React state directly mutate না করে নতুন object তৈরি করা ভালো।

✗ Avoid:

```
user.age = 32;

setUser(user);
```

এখানে একই object mutate করা হচ্ছে।

Better:

```
setUser((prev) => ({
  ...prev,
  age: 32
}));
```

আর dynamic key হলে:

```
setUser((prev) => ({
  ...prev,
  [key]: value
}));
```

18. Computed Property + Shorthand

ধরো:

```
const name = "Shariar";
const key = "username";
```

আমরা লিখতে পারি:

```
const user = {  
  [key]: name  
};
```

Result:

```
{  
  username: "Shariar"  
}
```

এখানে computed property ব্যবহার হয়েছে।

Shorthand property আলাদা concept:

```
const name = "Shariar";  
  
const user = {  
  name  
};
```

এটা equivalent:

```
const user = {  
  name: name  
};
```

19. Computed + Shorthand একসাথে

একই object-এ দুটোই ব্যবহার করা যায়।

```
const name = "Shariar";
const key = "role";
const role = "developer";

const user = {
  name,
  [key]: role
};
```

Result:

```
{
  name: "Shariar",
  role: "developer"
}
```

20. Object Property Keys কী হতে পারে?

JavaScript object-এর সাধারণ property keys মূলত:

- String
- Symbol

Number লিখলেও সেটি সাধারণ object property access-এর ক্ষেত্রে string key হিসেবে কাজ করে।

Example:

```
const obj = {
  1: "one",
  2: "two"
};

console.log(obj[1]);
```

Output:

```
one
```

Conceptually key:

```
"1"
```

হিসেবে কাজ করে।

21. Symbol as Computed Property

Symbol-ও computed property হিসেবে ব্যবহার করা যায়।

```
const id = Symbol("id");

const user = {
  [id]: 123
};
```

Access:

```
console.log(user[id]);
```

Output:

```
123
```

এটা advanced JavaScript-এর অংশ, তবে computed property বোঝার জন্য জানা ভালো।

22. Dynamic Keys from API Data

ধরো API থেকে data এসেছে:

```
const field = "username";
const value = "shariar";
```

আমরা dynamic object তৈরি করতে পারি:

```
const data = {
  [field]: value
};
```

Result:

```
{
  username: "shariar"
}
```

আর API-driven configuration:

```
const settingName = "theme";
const settingValue = "dark";

const settings = {
  [settingName]: settingValue
};
```

Result:

```
{
  theme: "dark"
}
```

23. Multiple Dynamic Properties

```
const key1 = "name";
const value1 = "Shariar";

const key2 = "role";
const value2 = "developer";

const user = {
  [key1]: value1,
  [key2]: value2
};
```

Result:

```
{  
  name: "Shariar",  
  role: "developer"  
}
```

24. Dynamic Keys inside Loops

Dynamic object তৈরি করতে loop-এর সাথেও computed properties/ bracket notation খুব useful।

```
const fields = ["name", "email", "age"];  
  
const user = {};  
  
for (const field of fields) {  
  user[field] = null;  
}  
  
console.log(user);
```

Result:

```
{  
  name: null,  
  email: null,  
  age: null  
}
```

এখানে:

```
user[field]
```

dynamic property access করছে।

25. Object from Array of Keys

আরও practical:

```
const keys = ["name", "email", "role"];

const user = {};

for (const key of keys) {
  user[key] = "unknown";
}
```

Result:

```
{
  name: "unknown",
  email: "unknown",
  role: "unknown"
}
```

26. Computed Property + Conditional Logic

Computed property-এর expression-এর মধ্যে conditional expression ব্যবহার করা যায়।

```
const isAdmin = true;

const user = {
  [isAdmin ? "admin" : "user"]: true
};
```

Result:

```
{
  admin: true
}
```

যদি:

```
const isAdmin = false;
```

তাহলে:

```
{
  user: true
}
```

তবে production code-এ unnecessarily complex computed key avoid করা ভালো।

27. Computed Property + Template Literal

Dynamic naming-এর ক্ষেত্রে template literal খুব useful।

```
const type = "user";

const data = {
  [`${type}Name`]: "Shariar",
  [`${type}Age`]: 31
};
```

Result:

```
{
  userName: "Shariar",
  userAge: 31
}
```

28. Computed Property + Spread for Immutable Updates

ধরো:

```
const state = {
  name: "Shariar",
  age: 31,
  role: "developer"
};

const key = "role";
const value = "senior developer";
```

Update:

```
const newState = {
  ...state,
  [key]: value
};
```

Result:

```
{
  name: "Shariar",
  age: 31,
  role: "senior developer"
}
```

Original `state` unchanged থাকে।

29. Dynamic Delete Pattern

ধরো dynamic key দিয়ে property remove করতে হবে।

Object destructuring + rest ব্যবহার করতে পারি।

```
const user = {
  name: "Shariar",
  age: 31,
  password: "secret"
};

const keyToRemove = "password";
```

একটি সাধারণ reusable function:

```
function removeProperty(object, key) {
  const { [key]: removed, ...rest } = object;

  return rest;
}
```

Usage:

```
const result = removeProperty(user, "password");
```

Result:

```
{  
  name: "Shariar",  
  age: 31  
}
```

এখানে:

একটি advanced computed destructuring pattern।

30. Computed Property in Destructuring

Normal destructuring:

```
const user = {  
  name: "Shariar",  
  age: 31  
};  
  
const { name } = user;
```

Dynamic property destructuring:

```
const key = "name";  
  
const {  
  [key]: value  
} = user;  
  
console.log(value);
```

Output:

```
Shariar
```

এখানে:

```
[key]
```

এর value `"name"`।

তাই effectively:

```
const {  
  name: value  
} = user;
```

31. Computed Destructuring Mental Model

এই code:

```
const key = "email";  
  
const {  
  [key]: value  
} = user;
```

ভাবতে পারো:

```
key  
  ↓  
"email"  
  ↓  
user.email  
  ↓  
value
```

অর্থাৎ dynamic property থেকে value বের করা হচ্ছে।

32. Nested Dynamic Update

Nested object থাকলে শুধু outer spread যথেষ্ট নয়।

ধরো:

```
const user = {
  name: "Shariar",
  address: {
    city: "Rangpur",
    country: "Bangladesh"
  }
};
```

আমরা dynamicভাবে city update করতে চাই:

```
const key = "city";

const updatedUser = {
  ...user,
  address: {
    ...user.address,
    [key]: "Dhaka"
  }
};
```

Result:

```
{
  name: "Shariar",
  address: {
    city: "Dhaka",
    country: "Bangladesh"
  }
}
```

33. কেন Nested Spread দরকার?

এটা:

```
const updatedUser = {  
  ...user,  
  address: {  
    [key]: "Dhaka"  
  }  
};
```

করলে পুরোনো `address`-এর অন্য properties হারিয়ে যাবে।

Original:

```
address: {  
  city: "Rangpur",  
  country: "Bangladesh"  
}
```

New:

```
address: {  
  city: "Dhaka"  
}
```

এখানে `country` চলে গেছে।

তাই:

```
address: {  
  ...user.address,  
  [key]: "Dhaka"  
}
```

ব্যবহার করতে হবে।

34. React Nested Dynamic State Update

ধরো:

```
const [user, setUser] = useState({
  name: "",
  address: {
    city: "",
    country: ""
  }
});
```

Dynamic address field update:

```
const handleAddressChange = (key, value) => {
  setUser((prev) => ({
    ...prev,
    address: {
      ...prev.address,
      [key]: value
    }
  }));
};
```

Usage:

```
handleAddressChange("city", "Rangpur");
```

অথবা:

```
handleAddressChange("country", "Bangladesh");
```

একই function দিয়ে দুই field update করা যায়।

35. Dynamic Form State — Full Example

```
import { useState } from "react";

function ProfileForm() {
  const [form, setForm] = useState({
    name: "",
    email: "",
    role: ""
  });

  const handleChange = (event) => {
    const { name, value } = event.target;

    setForm((prev) => ({
      ...prev,
      [name]: value
    }));
  };

  return (
    <form>
      <input
        name="name"
        value={form.name}
        onChange={handleChange}
      />

      <input
        name="email"
        value={form.email}
        onChange={handleChange}
      />

      <input
        name="role"
        value={form.role}
        onChange={handleChange}
      />
    </form>
  );
}
```

```
);  
}
```

এখানে মূল magic:

36. কেন `[name]` এত Important?

ধরো user name field-এ লিখল:

```
Shariar
```

তখন:

```
name = "name"  
value = "Shariar"
```

তাই:

হয়ে যায়:

```
name: "Shariar"
```

আবার email field:

```
name = "email"  
value = "shariar@example.com"
```

হয়ে যায়:

```
email: "shariar@example.com"
```

একই code।

37. Computed Property-এর সবচেয়ে গুরুত্বপূর্ণ Use Cases

Real-world development-এ এটি বেশি দেখা যায়:

1. Form state

```
setForm(prev => ({
  ...prev,
  [name]: value
}));
```

2. Dynamic configuration

```
const config = {
  [environment]: settings
};
```

3. API data transformation

```
const result = {
  [apiKey]: apiValue
};
```

4. Dynamic filters

```
const filters = {
  [filterName]: filterValue
};
```

5. Dynamic Redux/state updates

```
{
  ...state,
  [key]: value
}
```

6. Dynamic object creation

```
{  
  [key]: value  
}
```

38. Common Mistake — `obj.key` vs `obj[key]`

এটা খুব common interview question!

```
const key = "name";  
  
const user = {  
  name: "Shariar"  
};  
  
console.log(user.key);
```

Output:

```
undefined
```

কারণ এটা `"key"` property খুঁজছে।

Correct:

```
console.log(user[key]);
```

Output:

```
Shariar
```

Golden Rule

Variable-এর value দিয়ে property access করতে হলে `[]` ব্যবহার করো।

39. Common Mistake — Computed Brackets

ভুলে যাওয়া

✗ Wrong:

```
const key = "name";

const user = {
  key: "Shariar"
};
```

Result:

```
{
  key: "Shariar"
}
```

এখানে property-এর নাম literally `"key"`।

✓ Correct:

```
const user = {
  [key]: "Shariar"
};
```

Result:

```
{
  name: "Shariar"
}
```

40. Common Mistake — Nested State Mutation

✗ Avoid:

```
user.address.city = "Dhaka";

setUser(user);
```

এখানে existing object mutate করা হচ্ছে।

Better:

```
setUser((prev) => ({
  ...prev,
  address: {
    ...prev.address,
    city: "Dhaka"
  }
}));
```

Dynamic হলে:

```
setUser((prev) => ({
  ...prev,
  address: {
    ...prev.address,
    [key]: value
  }
}));
```

41. Common Mistake — Spread-এর Order

Object spread-এর order খুব important।

```
const user = {
  name: "Shariar"
};

const updated = {
  name: "Old",
  ...user
};
```

Result:

```
{  
  name: "Shariar"  
}
```

কারণ পরে আসা property জিতে যায়।

অন্যদিকে:

```
const updated = {  
  ...user,  
  name: "New"  
};
```

Result:

```
{  
  name: "New"  
}
```

Rule

Object literal-এ একই key পরে এলে আগের value overwrite করে।

42. Computed Property + Spread Order

```
const user = {  
  name: "Shariar"  
};  
  
const key = "name";  
  
const updated = {  
  ...user,  
  [key]: "Duke"  
};
```

Result:

```
{  
  name: "Duke"  
}
```

কারণ dynamic property পরে এসেছে।

43. Computed Properties কি Object Copy করে?

না।

এটা:

```
const key = "name";  
  
const user = {  
  [key]: "Shariar"  
};
```

শুধু dynamic property তৈরি করে।

এটা নিজে কোনো deep copy operation নয়।

Copy করতে চাইলে:

```
const copy = {  
  ...user  
};
```

তবে spread **shallow copy** তৈরি করে।

44. Computed Property vs Object Spread

দুটোর কাজ আলাদা।

Computed Property

Dynamic property তৈরি/access:

```
{  
  [key]: value  
}
```

Object Spread

Properties copy/merge:

```
{  
  ...user  
}
```

একসাথে

```
{  
  ...user,  
  [key]: value  
}
```

মানে:

পুরোনো object copy করো + dynamic key update করো।

45. Practical Example — Dynamic Product Update

```
const product = {
  name: "Laptop",
  price: 80000,
  stock: 10
};

function updateProduct(product, key, value) {
  return {
    ...product,
    [key]: value
  };
}
```

Usage:

```
const updated = updateProduct(
  product,
  "price",
  75000
);
```

Result:

```
{
  name: "Laptop",
  price: 75000,
  stock: 10
}
```

আর:

```
updateProduct(product, "stock", 8);
```

Result:

```
{
  name: "Laptop",
  price: 80000,
  stock: 8
}
```

একই function।

46. Practical Example — Dynamic Filter

```
const filterName = "category";
const filterValue = "electronics";

const filters = {
  [filterName]: filterValue
};
```

Result:

```
{
  category: "electronics"
}
```

আর:

```
const filterName = "price";
const filterValue = "50000";
```

Result:

```
{
  price: "50000"
}
```

47. Practical Example — Dynamic Object from Input

```
function createData(key, value) {  
  return {  
    [key]: value  
  };  
}  
  
console.log(createData("name", "Shariar"));  
console.log(createData("age", 31));  
console.log(createData("role", "frontend developer"));
```

Output:

```
{  
  name: "Shariar"  
}  
  
{  
  age: 31  
}  
  
{  
  role: "frontend developer"  
}
```

48. Practical Example — Dynamic Counters

ধরো বিভিন্ন category-এর count রাখতে চাই:

```
const category = "laptop";  
  
const counts = {  
  [category]: 1  
};
```

Result:

```
{  
  laptop: 1  
}
```

আর existing object update:

```
const counts = {  
  laptop: 2,  
  phone: 5  
};  
  
const key = "laptop";  
  
const updatedCounts = {  
  ...counts,  
  [key]: counts[key] + 1  
};
```

Result:

```
{  
  laptop: 3,  
  phone: 5  
}
```

49. Practical Example — Dynamic API Payload

```
const fieldName = "username";  
const fieldValue = "shariar";  
  
const payload = {  
  [fieldName]: fieldValue,  
  timestamp: Date.now()  
};
```

Result conceptually:

```
{
  username: "shariar",
  timestamp: 1234567890
}
```

এভাবে API payload dynamically তৈরি করা যায়।

50. Practical Example — Dynamic Object Keys with IDs

ধরো user ID:

```
const userId = "user_101";

const userData = {
  [userId]: {
    name: "Shariar",
    role: "developer"
  }
};
```

Result:

```
{
  user_101: {
    name: "Shariar",
    role: "developer"
  }
}
```

এটা cache, lookup table, normalized data ইত্যাদিতে useful।

51. Important Mental Model

Computed property দেখলে:

```
{  
  [something]: value  
}
```

নিজেকে জিজ্ঞেস করবে:

`something` evaluate করলে কী value পাওয়া যাবে?

Example:

```
const key = "email";  
  
const obj = {  
  [key]: "hello"  
};
```

Mental process:

```
[key]  
  ↓  
key evaluate  
  ↓  
"email"  
  ↓  
email: "hello"
```

52. Another Mental Model — Dynamic Access

যখন দেখবে:

```
object[key]
```

ভাববে:

```
key variable-এর value বের করো  
  ↓  
সেই value-কে property name হিসেবে ব্যবহার করো  
  ↓  
object থেকে property access করো
```

Example:

```
const key = "price";

product[key];
```

মান্নে:

```
product["price"];
```

53. Quick Comparison Table

Pattern	কাজ
<code>obj.name</code>	Static property access
<code>obj["name"]</code>	Static string key access
<code>obj[key]</code>	Dynamic property access
<code>{ name: value }</code>	Static property creation
<code>{ [key]: value }</code>	Dynamic property creation
<code>{ ...obj }</code>	Shallow copy
<code>{ ...obj, key: value }</code>	Static update
<code>{ ...obj, [key]: value }</code>	Dynamic update

54. Output Practice

Question 1

```
const key = "name";

const user = {
  [key]: "Shariar"
};

console.log(user);
```

Output কী?

Question 2

```
const key = "name";

const user = {
  name: "Shariar"
};

console.log(user[key]);
```

Question 3

```
const key = "name";

const user = {
  key: "Shariar"
};

console.log(user);
```

Question 4

```
const user = {
  name: "Shariar",
  age: 31
};

const key = "age";

const updated = {
  ...user,
  [key]: 32
};

console.log(updated);
```

Question 5

```
const type = "user";

const data = {
  [`${type}Name`]: "Shariar"
};

console.log(data);
```

55. Coding Challenge 1 — Dynamic Object Creator

একটি function লিখো:

```
createObject(key, value)
```

যেটা return করবে:

```
{
  [key]: value
}
```

Example:

```
createObject("name", "Shariar");
```

Expected:

```
{
  name: "Shariar"
}
```

56. Coding Challenge 2 — Dynamic Update

Function:

```
updateObject(object, key, value)
```

যেটা original object mutate না করে নতুন object return করবে।

Example:

```
const user = {  
  name: "Shariar",  
  age: 31  
};  
  
updateObject(user, "age", 32);
```

Expected:

```
{  
  name: "Shariar",  
  age: 32  
}
```

57. Coding Challenge 3 — Dynamic Form Handler

এই state:

```
const form = {  
  name: "",  
  email: "",  
  password: ""  
};
```

এমন একটি function লিখো:

```
updateForm(form, fieldName, value)
```

যেটা return করবে নতুন updated object।

Example:

```
updateForm(form, "email", "test@example.com");
```

Expected:

```
{  
  name: "",  
  email: "test@example.com",  
  password: ""  
}
```

58. Coding Challenge 4 — Remove Dynamic Property

Function লিখো:

```
removeProperty(object, key)
```

যেটা original object mutate না করে specified property remove করবে।

Example:

```
const user = {  
  name: "Shariar",  
  age: 31,  
  password: "123"  
};  
  
removeProperty(user, "password");
```

Expected:

```
{  
  name: "Shariar",  
  age: 31  
}
```

59. Interview Questions

Q1. Computed property name কী?

Object literal-এর property name expression/variable দিয়ে dynamically তৈরি করার syntax।

```
const key = "name";

const obj = {
  [key]: "Shariar"
};
```

Q2. `obj.key` এবং `obj[key]` এর পার্থক্য কী?

```
obj.key
```

literal `"key"` property access করে।

```
obj[key]
```

`key` variable-এর value অনুযায়ী property access করে।

Q3. `{ key: value }` এবং `{ [key]: value }` কি একই?

না।

```
const key = "name";

const obj1 = {
  key: "Shariar"
};
```

Result:

```
{
  key: "Shariar"
}
```

অন্যদিকে:

```
const obj2 = {  
  [key]: "Shariar"  
};
```

Result:

```
{  
  name: "Shariar"  
}
```

Q4. React form-এ `[name]: value` কেন ব্যবহার করা হয়?

কারণ input-এর `name` attribute অনুযায়ী dynamic property update করা যায়।

```
setForm(prev => ({  
  ...prev,  
  [name]: value  
}));
```

Q5. Computed property কি object-এর deep copy তৈরি করে?

না।

Computed property শুধু dynamic property তৈরি/access করে।

Q6. Object spread এবং computed property একসাথে কেন ব্যবহার করা হয়?

Immutable dynamic update-এর জন্য:

```
{  
  ...oldObject,  
  [key]: newValue  
}
```

Q7. Nested dynamic update কীভাবে করবে?

```
{
  ...state,
  nested: {
    ...state.nested,
    [key]: value
  }
}
```

60. Common Mistakes — Final List

✗ Mistake 1

```
obj.key
```

যখন `key` একটি variable।

✓ Correct

```
obj[key]
```

✗ Mistake 2

```
{
  key: value
}
```

যখন dynamic key দরকার।

✓ Correct

```
{
  [key]: value
}
```

✗ Mistake 3

React state mutate করা:

```
state[key] = value;
```

✓ Better

```
setState(prev => ({
  ...prev,
  [key]: value
}));
```

✗ Mistake 4

Nested object update-এ inner object copy না করা।

✓ Correct

```
{
  ...state,
  nested: {
    ...state.nested,
    [key]: value
  }
}
```

61. Quick Revision

```
const key = "name";
```

Dynamic access

```
user[key];
```

Dynamic property creation

```
const user = {  
  [key]: "Shariar"  
};
```

Dynamic update

```
const updated = {  
  ...user,  
  [key]: "Duke"  
};
```

Dynamic destructuring

```
const {  
  [key]: value  
} = user;
```

React dynamic state update

```
setState(prev => ({  
  ...prev,  
  [key]: value  
}));
```

Nested dynamic update

```
setState(prev => ({  
  ...prev,  
  nested: {  
    ...prev.nested,  
    [key]: value  
  }  
}));
```

62. Final Mental Model

Object-এর property যদি আগে থেকেই জানা থাকে:

```
user.name
```

Property যদি variable-এর মধ্যে থাকে:

```
const key = "name";

user[key];
```

Object বানানোর সময় dynamic key দরকার:

```
const obj = {
  [key]: value
};
```

Existing object immutableভাবে update করতে:

```
const updated = {
  ...obj,
  [key]: value
};
```

Nested object হলে:

```
const updated = {
  ...obj,
  nested: {
    ...obj.nested,
    [key]: value
  }
};
```

এই ৫টা pattern ভালোভাবে আয়ত্ত করলে **computed properties-এর core concept practically mastered!**

63. 🔥 Golden Rules

1. `obj.key` → `"key"` নামে property।
 2. `obj[key]` → `key` variable-এর value অনুযায়ী property।
 3. `{ key: value }` → static `"key"` property।
 4. `{ [key]: value }` → dynamic property।
 5. `[expression]` → expression evaluate করে property name তৈরি হয়।
 6. `...object` → shallow copy।
 7. `{ ...object, [key]: value }` → immutable dynamic update।
 8. Nested object update করলে nested level-ও spread করতে হয়।
 9. React state directly mutate না করে নতুন object তৈরি করা ভালো।
 10. Dynamic form handling-এ `[name]: value` অত্যন্ত গুরুত্বপূর্ণ pattern।
-



Section 7 — Final Mastery Checklist

তুমি যদি এগুলো নিজে explain ও code করতে পারো, তাহলে **Objects section strong**:

- ☐ Object literal
- ☐ `new Object()`
- ☐ Dot notation
- ☐ Bracket notation
- ☐ Add / update / delete property
- ☐ Nested objects
- ☐ `Object.keys()`
- ☐ `Object.values()`
- ☐ `Object.entries()`
- ☐ `Object.assign()`
- ☐ Object spread
- ☐ Shallow copy
- ☐ Object destructuring
- ☐ Destructuring defaults
- ☐ Destructuring renaming
- ☐ Nested destructuring
- ☐ Rest pattern
- ☐ Shorthand properties
- ☐ Shorthand methods
- ☐ Computed property names
- ☐ Dynamic property access

- ☐ Dynamic object creation
- ☐ Computed + spread
- ☐ Dynamic immutable update
- ☐ Dynamic React state update
- ☐ Nested dynamic update

Object section-এর সবচেয়ে important advanced pattern: 

```
> {  
  > ...oldObject,  
  > [dynamicKey]: newValue  
  > }  
>
```

React, forms, API data, filters, configuration, Redux/state management—সব জায়গাতেই এই pattern বারবার দেখতে পাবে।